

Selenium Java – PDF Download & Validation Exercise

A step-by-step practice exercise using Selenium WebDriver, ChromeOptions, Java File handling, PDFBox, and TestNG assertions.

Exercise Goal

Automate a scenario where Selenium opens a webpage, downloads a PDF, verifies that the file was downloaded successfully, opens the PDF using PDFBox, extracts its text, and validates expected data.

Final Automation Flow

Step	Action
1	Open the PDF download webpage using Selenium
2	Configure ChromeOptions for automatic downloads
3	Set the download directory
4	Locate and click the PDF download link/button
5	Wait until the PDF download is complete
6	Verify the PDF file exists
7	Verify the file size is greater than 0
8	Open the downloaded PDF using PDFBox
9	Validate the PDF page count
10	Extract PDF text using PDFTextStripper
11	Validate expected PDF data using assertions
12	Close the PDF and browser

Step 1 – Prepare the Maven Project

Use your existing Selenium Java Maven project. Create a package such as **seleniumPart1.Test0021PDFValidation** and create a class named **PDFDownloadTest**.

Step 2 – Add PDFBox

Add Apache PDFBox to pom.xml. PDFBox is responsible for reading and extracting text from the downloaded PDF; Selenium itself does not validate PDF content.

```
<dependency>
  <groupId>org.apache.pdfbox</groupId>
  <artifactId>pdfbox</artifactId>
  <version>3.0.6</version>
</dependency>
```

Step 3 – Choose a PDF Download Page

Use a webpage that contains a downloadable PDF. The page should have a stable download link/button. For a repeatable exercise, keep a known expected filename and known expected text in the PDF.

Step 4 – Configure the Download Directory

Create a Downloads directory in the project and construct its path dynamically with `System.getProperty("user.dir")`. Do not hard-code your personal Windows path.

```
String downloadPath = System.getProperty("user.dir") + "\\Downloads";
```

Step 5 – Configure ChromeOptions

Set Chrome preferences so the browser downloads automatically without displaying the download prompt. Also configure PDFs to open externally so Chrome does not simply display the PDF in its built-in viewer.

```
download.default_directory → downloadPath  
download.prompt_for_download → false  
download.directory_upgrade → true  
plugins.always_open_pdf_externally → true
```

Step 6 – Launch Selenium and Click Download

Launch Chrome with the configured ChromeOptions, open the target URL, locate the PDF download element, and click it.

```
driver.get("YOUR_PDF_DOWNLOAD_URL");  
driver.findElement(By.id("downloadPdf")).click();
```

Step 7 – Wait for the Download

Do not immediately assume the file is ready after clicking. Downloads are asynchronous. Check repeatedly until the expected file appears, with a reasonable timeout.

Also consider checking for a temporary Chrome download such as .crdownload when making the wait more robust.

Step 8 – Verify the File

Create a java.io.File object for the expected PDF and assert that it exists. Then verify that its length is greater than zero.

```
Assert.assertTrue(pdfFile.exists(), "PDF was not downloaded");  
Assert.assertTrue(pdfFile.length() > 0, "PDF is empty");
```

Step 9 – Open the PDF with PDFBox

Use PDFBox Loader.loadPDF(pdfFile) to open the downloaded file.

```
PDDocument document = Loader.loadPDF(pdfFile);
```

Step 10 – Validate Page Count

Read the number of pages and validate that the PDF contains the expected number of pages, or at minimum that it contains at least one page.

```
int pageCount = document.getNumberOfPages();  
Assert.assertTrue(pageCount > 0, "PDF has no pages");
```

Step 11 – Extract PDF Text

Create a PDFTextStripper and extract the text into a Java String.

```
PDFTextStripper stripper = new PDFTextStripper();  
String pdfText = stripper.getText(document);
```

Step 12 – Validate Business Data

Validate important values from the PDF instead of only checking that the file exists. For example, if the PDF contains a purchase report, validate the title, book name, date, amount, and location.

```
Assert.assertTrue(pdfText.contains("Selenium"));  
Assert.assertTrue(pdfText.contains("29-Jul-2019"));  
Assert.assertTrue(pdfText.contains("350"));  
Assert.assertTrue(pdfText.contains("Africa"));
```

Step 13 – Close Resources

Close the PDF document and quit the browser, preferably using try/finally so cleanup happens even when an assertion or other exception occurs.

Recommended Test Structure

Layer	Responsibility
Selenium	Open webpage and click PDF download
ChromeOptions	Control download behavior and location
Java File	Check downloaded file and file size
PDFBox	Open PDF, count pages, extract text
TestNG	Assert expected PDF data

What You Should Be Able to Explain in an Interview

- Why ChromeOptions are required for automatic PDF downloads.
- Why the download should be verified instead of using a fixed Thread.sleep alone.
- Why Selenium cannot directly validate PDF content.
- How PDFBox opens a PDF and extracts text.
- How to validate PDF content using TestNG assertions.
- How to use System.getProperty("user.dir") instead of hard-coded file paths.
- Why PDF resources and the WebDriver should be closed after the test.

Target outcome: Selenium downloads the PDF, Java verifies the file, PDFBox reads it, and TestNG validates the actual PDF data.